

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería

Departamento de Ciencias de la Computación

CC3084 – Redes

Sección 20

Ing. Jorge Yass



Laboratorio 3:

Protocolos de Enrutamiento

Vianka Vanessa Castro Ordoñez - 23201

Ricardo Arturo Godínez Sánchez - 23247

GUATEMALA, 13 de agosto de 2026

Vann06/Redes at Lab3

Introducción

El laboratorio consistió en diseñar e implementar una red simulada de routers capaces de aprender su topología de forma dinámica mediante un protocolo Link State, construir tablas de enrutamiento y reenviar mensajes por la ruta de menor costo. La guía del curso establece que cada nodo conoce inicialmente únicamente a sus vecinos y que la red debe operar mediante sockets, con pruebas locales y posteriormente sobre una red privada con Tailscale.

La solución se dividió en dos planos que se ejecutan de manera concurrente. El plano de control se encarga del descubrimiento de vecinos, la propagación de anuncios de estado de enlace (LSA), la construcción de la base de datos de estado de enlace (LSDB) y el cálculo de rutas. El plano de datos utiliza la tabla de enrutamiento generada para reenviar mensajes entre un cliente ATM y un servidor bancario, agregando detección y corrección de errores mediante Hamming(7,4).

Arquitectura

La implementación está desarrollada en Python y organizada en módulos independientes para configuración, transporte, protocolo, control, datos y endpoints. Cada router abre un servidor TCP y mantiene hilos separados para el envío de HELLO, la vigilancia de vecinos y la reemisión de LSA. Este diseño permite que el routing y el forwarding se mantengan activos simultáneamente.

Componente	Responsabilidad principal
Plano de control	HELLO, temporización de vecinos, LSA, flooding, LSDB, Dijkstra y generación de CSV.
Plano de datos	TTL, lista de saltos, consulta de la tabla CSV y reenvío hacia el siguiente salto o endpoint.
Capa de enlace lógico	Serialización de tramas y codificación/decodificación Hamming(7,4).
Endpoints	ATM como cliente y BANK como servidor; ambos se conectan a un router gateway.

Topología

La topología utilizada contiene nueve routers (A-I) y once enlaces ponderados. Los costos se mantienen simétricos y se leen desde el archivo de configuración compartido. El ATM utiliza A como puerta de enlace y el servidor BANK utiliza E.

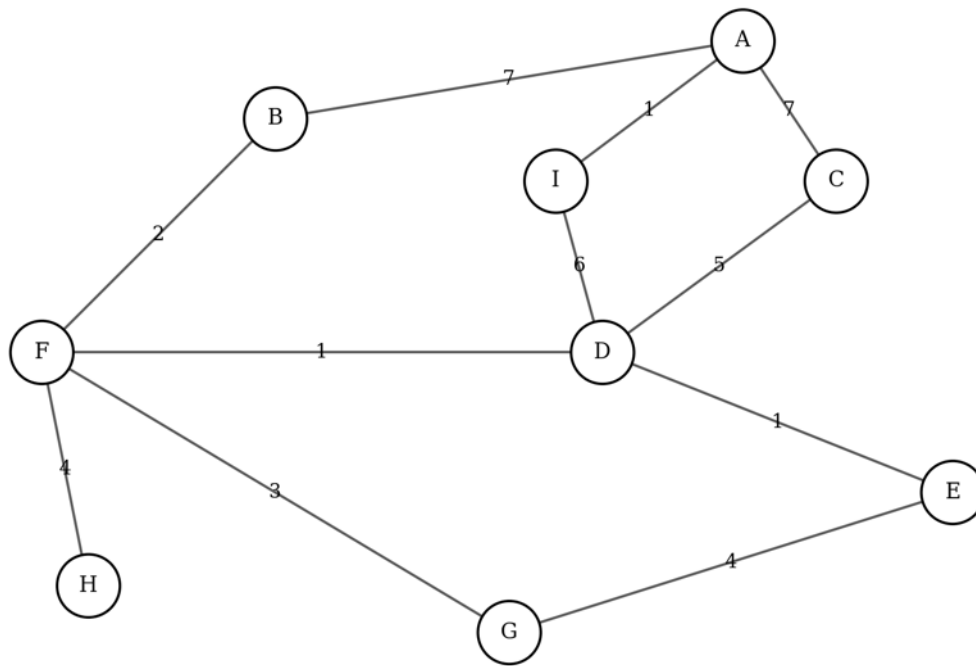


Figura 1. Topología de routers y costos de enlace utilizada en el laboratorio. Fuente: elaboración propia con base en la configuración del proyecto.

Enlace	Costo	Enlace	Costo
A-B	7	A-I	1
A-C	7	B-F	2
I-D	6	C-D	5
F-D	1	F-H	4
F-G	3	D-E	1
E-G	4		

Algoritmos utilizados e implementación

3.1 Descubrimiento de vecinos y Link State

Cada router envía paquetes HELLO a sus vecinos configurados cada 5 segundos. Un vecino se considera activo al recibirse su HELLO y se considera caído si transcurren 15 segundos sin

recibir uno nuevo. Cuando cambia el conjunto de vecinos activos, el router genera un LSA propio con un número de secuencia creciente.

Los LSA contienen el origen, número de secuencia, enlaces activos, nodo que reenvía el paquete y un TTL inicial de 8. Al recibir un LSA, el router solo acepta versiones con un número de secuencia mayor al previamente almacenado. Después actualiza su LSDB, recalcula rutas y reenvía el anuncio a todos sus vecinos, excepto al emisor inmediato. El TTL disminuye en cada salto para limitar la propagación.

Adicionalmente, el router vuelve a emitir su LSA cada 20 segundos. Esta decisión mejora la autorrecuperación de la red cuando una copia de un anuncio se pierde durante el arranque concurrente de varios procesos.

3.2 Dijkstra y construcción de la tabla

A partir de la LSDB se construye un grafo ponderado. La función de rutas mínimas utiliza una cola de prioridad (heap) para seleccionar el nodo con menor distancia acumulada y relajar sus enlaces. Para cada destino se reconstruye el primer salto desde el origen. El resultado se escribe en un archivo CSV por router con las columnas destino, siguiente_salto, costo, ip y puerto.

El uso de una cola de prioridad mantiene el cálculo eficiente para grafos dispersos; con esta estructura, la complejidad esperada es $O((V + E) \log V)$, donde V representa los nodos y E los enlaces.

3.3 Hamming(7,4)

La capa de datos aplica Hamming(7,4) con paridad par. Cada byte UTF-8 se divide en dos nibbles de 4 bits. Cada nibble $d_1 d_2 d_3 d_4$ se transforma en un bloque de 7 bits con la distribución $p_1 p_2 d_1 p_3 d_2 d_3 d_4$. En recepción se calcula el síndrome; si identifica una posición válida, se invierte ese bit y luego se extraen los cuatro bits de datos. Por cada byte original se transmiten 14 bits codificados.

3.4 Formato de tramas y reenvío

La propuesta de interoperabilidad del grupo distingue dos tipos de trama: J| para JSON UTF-8 en claro y H| para una cadena de bits codificada con Hamming(7,4). Los mensajes HELLO y LSA utilizan el formato de control, mientras que los mensajes de datos pueden enviarse codificados una vez que la red ha convergido.

```
{ "type": "message", "from": "ATM", "to": "BANK", "ttl": 16, "hops": [], "payload": {...}
}
```

En cada router, el plano de datos valida destino, TTL y lista de saltos. El paquete se descarta si el router actual ya aparece en hops, si el TTL se agotó o si no existe una ruta. En caso contrario, se disminuye el TTL, se agrega el router actual a hops y se consulta el CSV para obtener la IP y el puerto del siguiente salto. El payload no es interpretado por los routers.

Flujo de operación

1. Cada router inicia su servidor TCP y comienza a enviar HELLO a sus vecinos configurados.
2. Los vecinos que responden pasan a estado activo y provocan la generación de un nuevo LSA.
3. Los LSA se propagan por flooding y cada router construye progresivamente su LSDB.
4. Con la información disponible, cada router ejecuta Dijkstra y genera su tabla de enrutamiento CSV.
5. El ATM se conecta a su gateway A y envía un mensaje destinado a BANK.
6. Cada router consulta únicamente el destino, actualiza TTL/hops y reenvía al siguiente salto.
7. El router E entrega el mensaje al servidor BANK; la respuesta recorre la red de regreso hacia ATM.
8. Si se usa Hamming, cada router decodifica, corrige cuando corresponde y vuelve a codificar antes de reenviar.

Resultados

Las pruebas documentadas en el repositorio verificaron tanto componentes unitarios como el flujo completo de la aplicación. El conjunto de pruebas automatizadas reporta 27 de 27 casos exitosos. Además, se ejecutó una prueba extremo a extremo con nueve routers, BANK y ATM como procesos independientes.

```
Windows PowerShell
X + v

Cliente ATM: python src/endpoints/atm_cliente.py [--hamming]
Bajar todo: scripts/detener_red.sh
Ver un log: tail -f logs/A.log
PS C:\Users\richi\Documents\2026_S2_Local\Redes\Redes> bash scripts/levantar_red.sh
[levantar_red] router A -> PID 441
[levantar_red] router B -> PID 442
[levantar_red] router C -> PID 443
[levantar_red] router D -> PID 444
[levantar_red] router E -> PID 445
[levantar_red] router F -> PID 446
[levantar_red] router G -> PID 447
[levantar_red] router H -> PID 448
[levantar_red] router I -> PID 449
[levantar_red] esperando convergencia (~12s)...
[levantar_red] listo. Tabla de A:
destino,siguiente_salto,costo,ip,puerto
B,B,7,127.0.0.1,6002
C,C,7,127.0.0.1,6003
D,I,7,127.0.0.1,6009
E,I,8,127.0.0.1,6009
F,I,8,127.0.0.1,6009
G,I,11,127.0.0.1,6009
H,I,12,127.0.0.1,6009
I,I,1,127.0.0.1,6009

Cliente ATM: python src/endpoints/atm_cliente.py [--hamming]
Bajar todo: scripts/detener_red.sh
Ver un log: tail -f logs/A.log
PS C:\Users\richi\Documents\2026_S2_Local\Redes\Redes> |
```

```
Windows PowerShell
X + v

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\richi\Documents\2026_S2_Local\Redes\Redes> python src/endpoints/banco_servidor.py
[BANCO] escuchando en 127.0.0.1:7002 (gateway E)
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\richi\Documents\2026_S2_Local\Redes\Redes> python src/endpoints/atm_cliente.py
[CAJERO] escuchando respuestas en 127.0.0.1:7001 (gateway A)

Número de tarjeta: 23201
PIN: 6767
>> Autenticación exitosa

--- CAJERO ---
1) Retirar dinero
2) Enviar mensaje
3) Salir
Elige una opción: 1
Monto a retirar: 1
>> Toma tus $1.00
>> Saldo restante: $1599.00

--- CAJERO ---
1) Retirar dinero
2) Enviar mensaje
3) Salir
Elige una opción:
```

Prueba	Observaciones
Dijkstra	La ruta de A hacia E utiliza como siguiente salto I y tiene costo total 8.
Hamming(7,4)	Codificación/decodificación UTF-8 y corrección de un bit por bloque.
Framing	Round-trip de tramas JI y HJ; validación de tramas mal formadas.
Forwarding	Reenvío entre routers, entrega a endpoint y consulta del CSV.
Manejo de ciclos	Descarta si el router aparece previamente en hops.
TTL	Descarta paquetes con TTL agotado sin detener el proceso.
Flujo ATM-BANK	Login/retiro y respuesta verificados en una ejecución extremo a extremo.
Datos con Hamming	Un retiro enviado con codificación Hamming llegó al BANK y obtuvo respuesta.

Discusión

La separación entre plano de control y plano de datos simplifica el diseño esto es debido a que la lógica de routing puede actualizar la topología y las tablas sin mezclar las decisiones de forwarding con los mensajes de la aplicación. Por ejemplo en el uso de archivos CSV como interfaz entre ambos planos también facilita inspeccionar las rutas durante las pruebas.

Por otro lado, el uso conjunto de número de secuencia y TTL resulta importante para el flooding. El número de secuencia evita reprocesar anuncios viejos o duplicados, mientras el TTL introduce una segunda barrera contra propagación indefinida. En el plano de datos, la lista hops cumple una función equivalente para detectar bucles de ruteo.

Durante la validación se identificó que emitir LSA únicamente cuando cambia un vecino no garantizaba que todos los routers recuperaran un anuncio perdido. El refresco periódico de 20 segundos corrige este problema práctico y permite que la red vuelva a completar su LSDB sin esperar otro cambio de topología.

La configuración actual usa 127.0.0.1 para las pruebas locales. Para la prueba distribuida se deben reemplazar esas direcciones por las IP de Tailscale asignadas a cada integrante y confirmar que las tres parejas utilicen exactamente el mismo formato de tramas, nombres de campos, puertos y costos. Este paso es indispensable para garantizar interoperabilidad entre implementaciones distintas.

Conclusiones

Logramos implementar correctamente un protocolo Link State capaz de descubrir vecinos activos, propagar LSA, construir una LSDB y calcular rutas mínimas con Dijkstra.

La separación entre routing y forwarding permitió construir una arquitectura modular, en la que las tablas CSV funcionan como interfaz clara entre el plano de control y el plano de datos. Hamming(7,4) permitió integrar detección y corrección de errores a los mensajes de datos sin modificar la lógica interna de los routers sobre el payload.

Las pruebas automatizadas y la ejecución extremo a extremo validaron rutas, reenvío, corrección de errores, control de TTL y ciclos, además del flujo ATM-BANK.

Comentarios finales

Podríamos decir que un algoritmo de rutas mínimas por sí solo no es suficiente para construir un sistema de enrutamiento. La calidad del resultado depende también de cómo se descubre la vecindad, cómo se sincroniza el estado de la red, cómo se manejan anuncios duplicados o perdidos y cómo se protege el plano de datos frente a ciclos y errores de transmisión. La combinación de estos mecanismos nos permitió obtener una solución modular y verificable.